Argon MVP Specification: Local Analysis Engine

Oct 2, 2026 · @na

1 · Purpose & scope

The MVP is a local command-line tool that runs Argon's complete scoring engine on JavaScript, TypeScript and Python files on disk. There is no GitHub App, server or database. It must prove three things before Phase 1 infrastructure is built: the counters measure what they claim, the rules can't be cheaply gamed, and the thresholds give sensible scores on real code.

This spec is built from two sources only. Decision counting, nesting weights, scopes and the per-function ranking come from *How Argon scores code*. The data models, flag rules, purity score and later phases come from the *Argon V1 Build Specification* ("master"; section numbers like §4.2 refer to it). Every finding from the loophole review (§2) is written into the section it changes.

In scope

- Parsing with ast-grep for `.js .jsx .mjs .cjs .ts .tsx .py`
- One-pass decision counting, nesting weights, scopes and the isolation tax
- Determinate outputs, async boundaries, internal imports and effective lines
- All three master rules (concentration, semantic path, fragmentation), the purity score and the check conclusion
- Per-function ranking, a calibration report and the PR-comment preview in the CLI
- Test fixtures, plus parity checks against radon and ESLint

Out of scope until later phases

Item	Master section	Phase
Webhook receiver, signature check, debounce	5.1	1
Worker processes and cancellation	5.2	1
Check Runs and PR comments	5.5	1

Item	Master section	Phase
Postgres schema and `db/repository.py`	3.1, 5.6	1
Dashboard REST API	6	2
Enterprise forwarding and cloud templates	7.1, 7.2	3
`.argonignore` and `// argon-skip`	7.3	3

The review in §2 also covers the Phase 1 infrastructure. Those fixes are recorded there so they're settled before Phase 1, but the MVP doesn't build them.

2 · Loophole review

The review found 30 issues: 11 ways an AI agent can game the score, 10 math and edge-case faults, 5 runtime traps in the engine and 4 infrastructure traps for Phase 1. The worst five are line packing (A4), self-stamped `@generated` markers (A3), padded return statements (A1), a syntax-error check that misses broken files (C1), and Check Runs left stuck after a worker crash (D3). Items marked *verified* were reproduced with ast-grep-py 0.45.3 on 2026-10-02. The rest follow directly from the rules as written. Each fix is specified in full in the section listed.

2.1 Ways an agent can game the score

#	Sev.	Loophole	How it's gamed	Fix	§
A1	High	Return padding	Append unreachable `return "p1"`, `return "p2"` lines after a real return. Each adds a distinct output, dividing the rating with no new branch. Re-wording returns (`return (x)` vs `return x`) does the same.	Count outputs per function; ignore statements after an unconditional `return` / `throw` / `raise`; cap outputs at branches + 1	9.1
A2	Low	`?.` calls are free	`fn?.()` runs code conditionally but scores 0, while `??` scores 1	Optional config switch to count optional calls; default off for ESLint parity	6.3

#	Sev.	Loophole	How it's gamed	Fix	§
A3	High	Self-stamped `@generated`	Put `// @generated` on line 1 of every file. All files are skipped and the score is 100.	Honor generated markers only on paths matching `generated_path_globs`	10.1
A4	High	Line packing	Write long lines. `line_count` and `lines_altered` shrink, dodging both the 800-line concentration rule and the rating.	Use effective lines: non-blank, non-comment, each line counted as ceil(chars / 100)	9.4
A5	High	Ratio dilution	Touch 19 trivial files around 1 bad one. The flag ratio drops to 5% and the overshoot average to a twentieth.	Weight flag ratio and overshoot by effective altered lines; compute altered lines ignoring whitespace	10.3
A6	Medium	Closures reset depth	Wrap each nesting level in an inline arrow, lambda or IIFE. Depth restarts at 0 in each, so the `nested` flag never fires.	The flag uses effective depth, carried through scopes	8.2
A7	Medium	Short-circuit control flow	`a && (b && (c && run()))` is a 3-deep nest but scores 3 flat points instead of 6	A boolean operator whose value is discarded is scored as a structure, with its right side as the body	7.1
A8	High	Imports the counter can't see	Use `@/utils/x` path aliases or Python absolute imports (the Python norm). The relative-only counter sees 0 internal imports, so fragmentation never fires.	Resolve every import against the scanned file set: relative paths, tsconfig `paths` / `baseUrl`, Python dotted modules	9.3
A9	Medium	Fragmentation dilution	Add isolated files with no imports so the PR-wide average falls below 4	Measure density on the largest connected group of files that import each other	10.2
A10	Medium	Test-name camouflage	(Raised by the B9 fix.) Skipping test files entirely would let code hide under `tests/`	Test files are scored; only `assert` statements in them are not counted	6.1

#	Sev.	Loophole	How it's gamed	Fix	§
A11	High	Nesting doesn't gate anything	The nesting score only feeds the ranking. A 6-deep nest and a flat 6-case switch get the same rating, so deep nests cost nothing at merge time.	Compute a nesting-based rating alongside the master formula; choose by calibration	10.4

2.2 Math and edge cases

#	Sev.	Problem	Effect	Fix	§
B1	High	File-level branches squared	Branches rise with file length, so the rating grows roughly with lines cubed. A Sep 28 prototype on the 337 JS files of npm's CLI flagged 83% at threshold 40 and scored the project 12/100.	Per-function ratings; threshold set by calibration, not by guess	10.4
B2	High	Overshoot saturates	Ratings land in the thousands, so `min(avg_overshoot, 30)` is nearly always 30, a flat −30	Log-scaled overshoot: 10 points per 10× over threshold, capped at 30	10.3
B3	Medium	Tax overflow	The tax multiplies through each level of nesting. Deep callback chains in minified code pass 10^{15}, beyond JSON's safe integer (2^{53}) and Postgres `INTEGER` (2^{31}).	Saturate scores at 1,000,000 and set `saturated`	8.3
B4	Medium	Depth off-by-one	Weights count depth from 0. The ranking shows "1 = flat". The flag says "3 or more levels". The flag's trigger is ambiguous.	Reported depth = internal depth + 1; `nested` when reported depth ≥ 3 (`if` in `if` in `if`)	8.4
B5	Medium	Base complexity unstated	radon and ESLint add +1 per function. Added to a file total, 50 trivial functions would score 50.	Counts never include a base; parity tests add 1 per function when comparing	6.1
B6	Low	Banker's rounding	Python's `round()` gives 47.5 → 48 but 46.5 → 46	`floor(x + 0.5)`	10.3
B7	Medium	Generators score as single-output	A function that only `yield`s has 0 returns, so its divisor is 1	Count `yield` values as outputs	9.1

#	Sev.	Problem	Effect	Fix	§
B8	Low	Divide-by-zero and empty runs	Outputs of 0, no files, 0 altered lines	Already guarded by `max(outputs, 1)` and the empty check; kept, with tests	12.2
B9	Medium	Tests explode the count	Every pytest `assert` scores +1, so test files dominate the flagged list	Don't count `assert` in test paths (see A10)	6.1
B10	Medium	One node kind, two meanings	*Verified:* a `match` guard and a comprehension `if` are both `if_clause`. Counting the kind blindly counts guards, which the rules exclude.	Count `if_clause` only when its parent is a comprehension	6.2

2.3 Runtime traps in the engine

#	Sev.	Trap	Effect	Fix	§
C1	High	Missing-node blindness	*Verified:* `find(kind="ERROR")` returned False for an unclosed brace (TS), an unclosed `(` (JS) and `def f(:` (Python). tree-sitter marks these with zero-width MISSING nodes, not ERROR.	Also treat any zero-width non-root node as an error	5.2
C2	High	Error recovery hides code	Code swallowed by an error node isn't counted, so syntax the grammar can't read (Flow types in `.js`, new proposals) scores low	Fail closed: above 5% error coverage the file is flagged `unparseable`	5.2
C3	Medium	Non-UTF-8 crash	`decode("utf-8")` raises on legacy encodings and aborts the whole run	Decode with `errors="replace"`; strip a BOM	5.1
C4	Medium	Huge files	Multi-megabyte bundles cost time and memory	Over 1 MB: not parsed, flagged `oversize` unless on a generated path	10.1
C5	Low	Symlink loops	Walking folders can loop forever or leave the repo	Don't follow symlinks; drop files whose real path is outside the scan root	11

2.4 Phase 1 infrastructure (recorded, not built in the MVP)

#	Sev.	Claim	Verdict	Fix for Phase 1
D1	Medium	PID reuse kills the wrong process (master 5.2)	**Partly valid.** Through the `multiprocessing.Process` handle it can't happen: a dead, unreaped child is a zombie whose PID Linux won't reuse, and once reaped, `terminate()` does nothing. The real risk is killing by the `worker_pid` stored in Postgres, for example from another web replica or after a restart. A replica also can't cancel a worker started by another replica.	Never signal a stored PID; cancel only through the in-process handle. Before `complete_check_run()`, the worker re-reads `active_analyses` and posts only if `current_head_sha` still equals its own SHA. `worker_pid` becomes information only.
D2	High	Webhook GitHub calls breach the 10-second limit (master 5.1)	**Valid, and worse than stated.** PyGithub is synchronous, so inside an `async def` route it blocks the event loop. Every webhook then waits behind every other webhook's GitHub calls, and rate-limit backoff can add seconds. GitHub does not automatically redeliver failed deliveries, so those events are lost.	The receiver verifies the signature, inserts the delivery into a Postgres queue table (unique on delivery ID), and returns 202. A separate dispatcher supersedes and creates Check Runs, using async HTTP with timeouts.

#	Sev.	Claim	Verdict	Fix for Phase 1
D3	High	In-memory registry leaks (master 5.2)	**Partly valid.** The registry is keyed by (repo, PR), so entries are overwritten and memory stays small. The real damage is elsewhere. Unjoined children pile up as zombies. A worker killed by out-of-memory leaves its `active_analyses` row at `running` and its Check Run `in_progress` forever, which blocks merging when the check is required.	A reaper runs on every enqueue and every 30 s: it joins dead processes and removes their entries. A non-zero exit completes the Check Run as `failure` titled "Argon crashed — re-run", then clears the row. Each worker gets a memory cap (`RLIMIT_AS`) and a 120 s time limit.
D4	Medium	Stale results after a missed cancel	(Found with D1.) If a cancel misses, an old worker can post over a newer one	Covered by the SHA re-check in D1

3 · Directory layout & dependencies

The MVP creates the files marked MVP. Phase 1 adds the others beside them, so nothing moves later.

```
argon/
├── pyproject.toml
├── argon/
|   ├── schemas.py              # MVP  data models (§4)
|   ├── cli.py                  # MVP  only place that touches disk or git
(§11)
|   ├── engine/
|   |   ├── config.py           # MVP  every tunable constant (§10.1)
|   |   ├── languages.py        # MVP  parse_source (§5)
|   |   ├── branches.py         # MVP  what counts as a decision (§6)
|   |   ├── nesting.py          # MVP  what raises depth (§7)
|   |   ├── scopes.py           # MVP  scope kinds, isolation tax (§8)
|   |   ├── ast_counters.py     # MVP  analyze_tree: the one-pass walk (§8.5,
§9)
|   |   ├── imports.py          # MVP  import resolution (§9.3)
```

```
|   |   ├── run.py              # MVP  rules, purity score, summary (§10)
|   |   └── forward.py          # Phase 3
|   ├── webhook/receiver.py     # Phase 1
|   ├── queue/analysis_queue.py # Phase 1
|   ├── github/poster.py        # Phase 1
|   └── db/repository.py        # Phase 1
└── tests/
    ├── fixtures/               # MVP  (§12.1)
    ├── parity/                 # MVP  radon and ESLint comparisons (§12.3)
    └── test_*.py               # MVP
```

Package	Version	Use
Python	3.12	Runtime
`ast-grep-py`	⩾ 0.45 (0.45.3 tested)	Parsing; ships prebuilt Rust binaries per platform
`pydantic`	⩾ 2	Data models and config
`pytest`	any	Tests (dev)
`radon`	6.0.1 exactly	Python parity tests (dev)
`eslint` (npm)	9.x	JS/TS parity tests (dev), `complexity` rule

ast-grep ships its grammars inside the wheel, so master §2.3's grammar compile step and its cross-architecture crash don't apply.

Import rule: nothing under `engine/` does I/O. It takes bytes and returns models, so the Phase 1 worker can call it unchanged. A test fails the build if `engine/` imports `os`, `pathlib`, `subprocess`, `socket` or an HTTP library.

4 · Data models (`schemas.py`)

All of master §3.2's `FileFinding` fields stay, with the same names. New fields are only added, so Phase 1 code written against master still works. Two fields keep their names but change meaning, as noted in the comments. `AuditResult` from master extends `EngineResult` in Phase 1.

```
# argon/schemas.py
from typing import Literal, Optional
from pydantic import BaseModel

FlagReason = Literal[
    "concentration_rule",   # master §4.1
    "semantic_path_logic",  # master §4.2
    "unparseable",          # §5.2, fail closed (C2)
    "oversize",             # §10.1, fail closed (C4)
]

class FunctionFinding(BaseModel):
    name: str                   # qualified: "Cart.total", "handler" (§8.1)
    start_line: int             # 1-based
    end_line: int
    score: int                  # own weighted decisions + taxed inner
scopes (§8)
    branches: int               # decisions written directly in this
function, no base
    max_depth: int              # reported, 1 = flat; effective depth (§8.2,
§8.4)
    effective_lines: int        # §9.4
    effective_lines_altered: int
    determinate_outputs: int    # raw distinct outputs (§9.1)
    outputs_effective: int      # min(raw, branches + 1), at least 1
    rating_master: float        # §10.4
    rating_nesting: float       # §10.4
    nested: bool                # max_depth >= 3
    saturated: bool = False     # score hit SCORE_CAP (B3)

class FileFinding(BaseModel):
    # master §3.2 fields
    filename: str               # POSIX, relative to scan root
    line_count: int             # physical lines, display only (rules use
effective_lines)
    async_boundary_count: int
    cyclomatic_branches: int    # flat count, whole file, no base (B5)
    lines_altered: int          # physical, display only
    determinate_outputs: int    # sum of functions' outputs_effective
    relative_import_count: int  # now ALL imports resolving to a scanned
file (A8)
    logic_complexity_rating: float  # the rating used for gating (§10.4)
    flagged: bool
```

```
    flag_reason: Optional[FlagReason] = None
    # added
    language: Literal["javascript", "typescript", "tsx", "python"]
    effective_lines: int
    effective_lines_altered: int
    nesting_score: int          # weighted, whole file incl. module level
    max_depth: int              # deepest effective decision in the file
    parse_error: bool
    error_coverage: float       # 0.0-1.0 share of bytes inside
ERROR/MISSING
    functions: list[FunctionFinding]

class SkippedFile(BaseModel):
    filename: str
    reason: Literal["generated_path", "vendored_path",
"unsupported_extension"]

class EngineResult(BaseModel):
    purity_score: int           # 0-100
    check_conclusion: Literal["success", "failure"]
    rating_mode: Literal["master", "nesting"]
    findings: list[FileFinding]
    flagged_file_count: int
    skipped_file_count: int
    skipped_files: list[SkippedFile]
    ignored_generated_markers: list[str]   # files whose marker was not
honored (A3)
    fragmentation_flagged: bool
    fragmentation_cluster: list[str]       # files in the triggering group
(§10.2)
    summary_markdown: str
```

When several flag reasons apply, `flag_reason` takes the first in this order: `oversize`, `unparseable`, `concentration_rule`, `semantic_path_logic`. Anonymous functions have no `FunctionFinding` of their own; their score rolls into the enclosing row (§8.1).

5 · Parsing (`engine/languages.py`)

`parse_source()` keeps the original design (ast-grep, one grammar per extension) with three fixes: it survives bad encodings (C3), detects MISSING nodes (C1), and measures how much of the file the parser failed to read (C2). The code below was run against the

three broken samples from §2.3, all of which it now catches. On clean code it reported an error in 1 of 1,054 npm JS files and 0 of 170 Python standard-library files.

5.1 Code

```
# argon/engine/languages.py
"""Rust-backed JavaScript, TypeScript, and Python parsing through ast-
grep."""
from __future__ import annotations
from dataclasses import dataclass
from pathlib import PurePosixPath
from ast_grep_py import SgNode, SgRoot

LANGUAGES = {
    ".js": "javascript", ".jsx": "javascript", ".mjs": "javascript", ".cjs":
"javascript",
    ".ts": "typescript", ".tsx": "tsx", ".py": "python",
}
SUPPORTED_EXTENSIONS = frozenset(LANGUAGES)

@dataclass(frozen=True)
class ParsedSource:
    root: SgNode
    language: str
    text: str                # decoded source; line math uses this
    has_error: bool
    error_coverage: float    # 0.0-1.0

def language_for(filename: str) -> str | None:
    return LANGUAGES.get(PurePosixPath(filename.replace("\\",
"/")).suffix.lower())

def parse_source(filename: str, source: bytes) -> ParsedSource:
    language = language_for(filename)
    if language is None:
        ext = PurePosixPath(filename.replace("\\", "/")).suffix.lower()
        raise ValueError(f"unsupported source extension: {ext or '<none>'}")
    # C3: never raise on bad bytes; a BOM would otherwise shift columns
    text = source.decode("utf-8", errors="replace").removeprefix("")
    root = SgRoot(text, language).root()
    error_chars = missing = 0
    stack = [root]
```

```
    while stack:
        node = stack.pop()
        r = node.range()
        if node.kind() == "ERROR":
            error_chars += r.end.index - r.start.index
            continue                      # nested errors are inside this
span
        if node is not root and r.start.index == r.end.index:
            missing += 1                  # C1: zero-width = MISSING node
        stack.extend(node.children())
    coverage = min(1.0, (error_chars + missing) / max(len(text), 1))
    return ParsedSource(root, language, text, error_chars + missing > 0,
coverage)
```

5.2 Rules

- **Why zero-width.** ast-grep-py exposes no `is_missing()`. When tree-sitter recovers by inventing a token (a closing `}` or `)`), that token has zero width. Valid code produced no zero-width nodes in any sample tested, including empty strings, empty blocks, TSX fragments and empty files.
- **Fail closed (C2).** If `error_coverage > config.max_error_coverage` (default 0.05), the file is not scored. It is flagged with `flag_reason="unparseable"` and counts as flagged in the purity score. Hiding code inside a parse error can then never lower a score.
- **Below the limit,** the file is scored normally, `parse_error=True` is set, and the CLI prints a warning.
- **`.ts` vs `.tsx`.** These need different grammars; JSX parsed as plain TS produces errors on every tag.
- **One walk.** The error scan above and `analyze_tree()` (§8.5) can be merged into one traversal later. They are separate here for clarity, and parsing 400 files with a full walk took 0.5 s in testing.

6 · Counting decisions (`engine/branches.py`)

Every decision adds +1 to the flat count, the master `cyclomatic_branches`. The same decisions feed the nesting-weighted score in §7. Flat counts match radon 6.0.1 (Python) and ESLint's `complexity` rule (JS/TS), apart from the differences listed in §6.4. Node kinds below were checked against ast-grep-py 0.45.3.

6.1 General rules

- **No base.** radon and ESLint start each function at 1. Argon counts decisions only, in functions and in files (B5). Parity tests add 1 per function before comparing.
- **Two kinds of decision.** A *structure* is every decision except a boolean operator; it is weighted by depth (§7). A *boolean operator* always weighs a flat +1, unless it is used as control flow (§7.1).
- **Asserts in tests (B9, A10).** In files matching `config.test_path_globs`, `assert` statements and the boolean operators inside them are not counted. Everything else in test files is counted. Test files are never skipped, so moving code under `tests/` doesn't hide it.

```
test_path_globs = ("**/test_*.py", "**/*_test.py", "**/conftest.py",
"**/tests/**",
                   "**/__tests__/**", "**/*.test.*", "**/*.spec.*")
```

6.2 Python

Construct	ast-grep kind	Counted
`if`, `elif`	`if_statement`, `elif_clause`	+1 each
`else` after `if`	`else_clause` with parent `if_statement`	No
`else` on a loop or `try`	`else_clause` with parent `for_statement`, `while_statement` or `try_statement`	+1
`for`, `async for`, `while`	`for_statement`, `while_statement`	+1
`except`, `except*`	`except_clause` (both forms)	+1 each
`try`, `finally`, `with`	`try_statement`, `finally_clause`, `with_statement`	No
`case`	`case_clause`	+1 each, except the first catch-all (below)
`match` guard `case 1 if x:`	`if_clause` with parent `case_clause`	No (B10)

Construct	ast-grep kind	Counted
`b if a else c`	`conditional_expression`	+1
`and`, `or`	`boolean_operator`	+1 each (`not` is `not_operator`: no)
`assert`	`assert_statement`	+1, plus its `and` / `or` (except in tests)
Comprehension clauses	`for_in_clause`, and `if_clause` with parent `list_comprehension`, `set_comprehension`, `dictionary_comprehension` or `generator_expression`	+1 each

A catch-all `case_clause` has no `if_clause` guard and a single `case_pattern` that is either `_` or a capture: one `dotted_name` holding exactly one `identifier`. `case Color.RED` has two identifiers, so it's a value pattern and counts.

6.3 JavaScript and TypeScript

Construct	ast-grep kind	Counted
`if`, `else if`	`if_statement` (an `else if` is an `if_statement` inside an `else_clause`)	+1 each
plain `else`	`else_clause`	No
`for`, `for…in`, `for…of`, `while`, `do…while`	`for_statement`, `for_in_statement` (covers `of`), `while_statement`, `do_statement`	+1
`catch`	`catch_clause`	+1
`try`, `finally`	`try_statement`, `finally_clause`	No
`case` / `default:`	`switch_case` / `switch_default`	+1 / No
`a ? b : c`	`ternary_expression`	+1
`&&`, `||`, `??`	`binary_expression` with that operator	+1
`&&=`, `||=`, `??=`	`augmented_assignment_expression` with that operator	+1

Construct	ast-grep kind	Counted
`?.`, default parameters	`optional_chain`, `assignment_pattern`	No (A2: see below)

A2 switch. `config.count_optional_calls` (default `False`) adds +1 for a `call_expression` whose callee contains an `optional_chain`, so `fn?.()` counts and `a?.b` doesn't. It stays off until calibration shows whether `?.` calls are being used to dodge the count.

6.4 Deliberate differences from radon and ESLint

Tool	Difference	Why
radon 6.0.1	`except*` counts +1 each; radon scores it 0	radon predates `except*`
radon 6.0.1	`and` / `or` inside `assert` count; radon skips them	An assert's condition is still a decision
ESLint	`?.` and default parameters don't count; ESLint counts both	They add no branching logic the author wrote
both	No +1 base per function	B5
both	`assert` in test files doesn't count	B9

7 · Nesting weight & depth (`engine/nesting.py`)

A structure at depth *d* costs *d* + 1, with depth starting at 0. A boolean operator costs a flat +1 at any depth. Depth rises by one only inside the body of a decision; conditions, chains and non-decision blocks stay at the same level.

```
if a:                # depth 0 → +1
    if b and c:      # depth 1 → +2, plus the `and` → +1
        if d:        # depth 2 → +3
            ...
# nesting score 7; flat count 4
```

Three nested `if` blocks score 1 + 2 + 3 = 6, where the flat count is 3.

7.1 Control-flow booleans (A7)

A boolean operator whose value is thrown away is control flow, not a condition. It is scored as a structure at its depth, and its right operand is its body at depth + 1. An operator is in a *discarded* position when:

- its parent is an `expression_statement`, or
- its parent is a `parenthesized_expression` that is itself in a discarded position, or
- it is the right operand of a control-flow boolean.

`ok && (b && (c && run()))` now scores 1 + 2 + 3 = 6, the same as three nested `if`s. `a && b && c && run()` parses left to right as `((a && b) && c) && run()`, so only the outer operator is control flow: 1 + 1 + 1 = 3, the same as `if (a && b && c) run()`. Assignments like `x = a && b` are untouched. The flat count is the same either way.

7.2 Which parts sit at which depth

For a decision at depth *d*. "Fields" are ast-grep field names, checked against 0.45.3.

Kind	Stays at *d*	Goes to *d* + 1
Python `if_statement`	`condition`; each `elif_clause` and `else_clause` child	`consequence`
Python `elif_clause` (weighted at *d*: a chain)	`condition`	`consequence`
Python `else_clause`	(weighted at *d* when on a loop or `try`)	`body`
Python `for_statement`, `while_statement`	`left`, `right`, `condition`; the `else_clause`	`body`
Python `try_statement`	`body`, `finally_clause`, each `except_clause`	(nothing)
Python `except_clause`	its exception expression	its `block` child (it has no `body` field)
Python `case_clause`	`case_pattern`, `guard`	`consequence`

Kind	Stays at *d*	Goes to *d* + 1
Python `conditional_expression` `p if q else r`	condition `q` (2nd named child); `r` too if `r` is itself a `conditional_expression` (a chain)	`p` (1st named child); `r` otherwise
JS `if_statement`	`condition`; an `if_statement` directly inside the `else_clause` (a chain)	`consequence`; any other `else_clause` body
JS `for_statement`, `for_in_statement`, `while_statement`, `do_statement`	`initializer`, `condition`, `increment`, `left`, `right`	`body`
JS `switch_case`	`value`	every child after `value`, not just the `body` field (which holds only the first statement)
JS `try_statement`	`body`, `finalizer`, `handler`	(nothing)
JS `catch_clause`	`parameter`	`body`
JS `ternary_expression`	`condition`; `alternative` if it is itself a `ternary_expression` (a chain)	`consequence`; `alternative` otherwise
Control-flow boolean (§7.1)	left operand	right operand
`with_statement`, `finally_clause`, plain blocks	everything	(nothing)

Two field traps: Python's `if_statement.alternative` returns only the first `elif`, and JS `switch_case.body` returns only the first statement. Always iterate the children.

7.3 Comprehensions

A comprehension scores like the loop it stands for. With *n* clauses, clause *i* (counting from 0) and everything inside it sits at depth *d* + *i*. The element expression (field `body`) is innermost, at *d* + *n*.

```
rows = [f(x) for a in b if c for x in a]      # 1 + 2 + 3 = 6

# identical score to:
for a in b:              # +1
    if c:                # +2
        for x in a:      # +3
            rows.append(f(x))
```

A comprehension inside a loop header (`for r in (p for p in ps if p):`) sits in the loop's `right` field, at the loop's depth *d*, so it isn't nested inside the loop. This matches execution: it runs once, before the loop.

8 · Scopes, the isolation tax & the walk (`engine/scopes.py`, `engine/ast_counters.py`)

Every function, method, lambda, arrow function and class is scored on its own, starting again from depth 0. The enclosing code then pays **inner_score × (1 + definition depth)**. That is the only way an inner score is counted, so nothing is counted twice. Functions defined flat (depth 0) cost exactly their own score, so ordinary functions and methods are unaffected.

```
def outer(a, b):
    if a:                    # outer, depth 0 → +1
        def inner():         # defined at depth 1 → outer pays inner × 2
            if b:            # inner, depth 0 → +1
                pass
# inner = 1; outer = 1 + 1 × 2 = 3     (flat count: 2)

def outer():
    for x in xs:             # +1
        if x:                # +2
            def inner():     # defined at depth 2 → pays inner × 3
                if y:        # +1
                    if z:    # +2   → inner = 3
                        pass
# outer = 1 + 2 + 3 × 3 = 12
```

8.1 Scope kinds and rows

Language	Scope node kinds
Python	`function_definition` (incl. `async def`), `lambda`, `class_definition`
JS/TS	`function_declaration`, `generator_function_declaration`, `function_expression`, `generator_function`, `arrow_function`, `method_definition`, `class_declaration`, `class`

Only **named** functions get a row in the ranking. A function is named when it is a declaration or method, or when an anonymous function is the direct value of a `variable_declarator`, an object `pair`, a class field, or a Python `assignment` to a plain name. The row takes that name, qualified by its class (`Cart.total`). Every other function is **anonymous**: callbacks, IIFEs, lambdas passed as arguments. Anonymous functions are still taxed into their parent as above, but they have no row, so a file with 200 `.map(x => …)` callbacks doesn't produce 200 rows. Classes get no row either, since a class's score is just the sum of its methods. Module-level code counts toward the file total but has no row.

8.2 Effective depth for the `nested` flag (A6)

Scores reset depth inside each scope, but the `nested` flag must not, or wrapping each level in a closure would hide any nest. Each scope carries a base depth B:

- The module has $B = 0$.
- A scope defined at local depth d inside a parent with base B_p gets $B = B_p + d$.
- A decision at local depth k has effective depth $B + k$.

Top-level functions and methods have $B = 0$, so their effective depth equals their local depth. A callback inside `if a:` that contains `if b:` has effective depth 1, the same as writing `if b:` directly inside `if a:`.

8.3 Saturation (B3)

The tax multiplies at each level, so deep callback chains grow without limit. Every score is clamped at `config.score_cap` (default 1,000,000) and the row gets `saturated=True`. A parent of a saturated scope is saturated too. Flat counts are additive and never need the cap.

8.4 Reported depth

Internally depth starts at 0. Every reported depth is internal + 1, so 1 means flat. A row is flagged `nested` when its reported `max_depth` is 3 or more: an `if` inside an `if` inside an `if` (B4). A row's `max_depth` covers its own decisions and those of its anonymous children.

8.5 The walk

`analyze_tree(parsed, cfg)` makes one pass with an explicit stack on the heap, never recursion. Long `a + b + …` chains, minified code and thousands of nested callbacks can't hit Python's call-stack limit.

```
stack = [Visit(root, depth=0)]
frames = [Frame(kind="module", base=0)]       # running totals per open scope
while stack:
    item = stack.pop()
    if isinstance(item, EndScope):           # every inner node is scored
        inner = frames.pop()
        parent = frames[-1]
        parent.score = cap(parent.score + inner.score * (1 + item.def_depth))
        parent.flat += inner.flat
        record_row_or_merge(inner, parent)   # named → row; anonymous → merge
depth stats
        continue
    node, d = item.node, item.depth
    if is_scope(node):
        frames.append(Frame(node, base=frames[-1].base + d))
        stack.append(EndScope(def_depth=d))
        push_children(stack, node, depth=0)  # inner scale restarts at 0
        continue
    if is_decision(node):                    # §6, incl. control-flow
booleans
        f = frames[-1]
        f.flat += 1
        f.score += 1 if is_plain_boolean(node) else d + 1
        f.max_depth = max(f.max_depth, f.base + d)
    for child, child_depth in child_depths(node, d):   # §7.2 table
        stack.append(Visit(child, child_depth))
```

The same pass collects the counters in §9. The file's `nesting_score` is the module frame's final score; `cyclomatic_branches` is its flat count.

8.6 Per-function ranking columns

Column	Meaning
`score`	the function's own weighted decisions plus the taxed scores of scopes nested inside it
`branches`	decisions written directly in this function and its anonymous children, no base
`depth`	its deepest effective decision, reported (1 = flat)
`lines`	effective lines (§9.4)
`flags`	`nested` (depth ≥ 3; flatten with early returns or helpers), `saturated`

Rows sort by score descending, then lines descending, then filename and start line, so output is identical across runs.

9 · Other per-file counters (`engine/ast_counters.py`, `engine/imports.py`)

All four are collected in the same walk as §8.5. Outputs and effective lines are kept per scope so every function can be rated on its own (§10.4).

9.1 Determinate outputs (A1, B7)

The divisor in the rating. It counts the distinct values a function can hand back, per scope.

Language	Output sources
Python	`return_statement`; `yield` (B7); a `lambda` body
JS/TS	`return_statement`; `yield_expression`; an `arrow_function` whose body is not a `statement_block`

1. **Key.** The whitespace-collapsed text of the returned expression with outer parentheses removed, or `<void>` for a bare `return` / `yield`. Duplicate keys count once, so `return None` in five places counts once.

2. **Skip dead code.** In any block, statements after an unconditional `return`, `throw` (JS) or `raise` (Python) are not visited for outputs.
3. **Cap.** `outputs_effective = max(1, min(raw, branches + 1))`, where `branches` is the function's flat count. A function with *b* decisions has at most *b* + 1 independent paths, so it can't claim more distinct exits. This blocks padding however the returns are written.
4. **Own scope only.** A callback's returns belong to the callback, not to the function containing it.

The file-level `determinate_outputs` is the sum of every row's `outputs_effective`, for display.

9.2 Async boundaries

Used by the concentration rule. A boundary is one independent listener, socket or consumer. Lists live in config so they can grow.

Language	Counts as a boundary	Deduplicated by
JS/TS	`call_expression` on a `member_expression` whose property is in `async_listener_methods` (`on`, `once`, `addListener`, `addEventListener`, `subscribe`, `consume`, `process`, `onSnapshot`)	the receiver's text: `socket.on("message")` and `socket.on("close")` are one
JS/TS	`new_expression` of a name in `async_constructors` (`WebSocket`, `WebSocketServer`, `EventSource`, `Worker`)	each site
JS/TS	assignment to `<x>.onmessage`	the receiver's text
Python	a `call` on an attribute in `py_async_listener_methods` (`subscribe`, `consume`, `basic_consume`, `on`, `add_listener`, `add_event_handler`)	the receiver's text

Language	Counts as a boundary	Deduplicated by
Python	a `decorator` whose last attribute is in `py_async_decorators` (`task`, `shared_task`, `websocket`, `on_event`, `subscriber`)	each site
Python	a `call` to a name in `py_async_constructors` (`KafkaConsumer`, `AIOKafkaConsumer`, `Consumer`, `WebSocketApp`)	each site

Master says to deduplicate "by call-site". Read literally, one socket with four handlers is four boundaries and trips the rule by itself. This spec deduplicates listeners by receiver instead (open question 3).

9.3 Internal imports (A8)

Counts the distinct files in the scan set that this file imports, however the import is written. It fills master's `relative_import_count` and the fragmentation graph (§10.2). The engine does no I/O, so the CLI reads `tsconfig.json` / `jsconfig.json` at the scan root and passes the alias map in through config.

Source	Collected from	Resolution
JS relative	`import_statement` and `export_statement` `source`; `require("…")`; dynamic `import("…")`	Join with the importer's folder. Try the path as written, then with `.ts .tsx .js .jsx .mjs .cjs`, then `/index` plus each. A specifier ending in `.js` also tries `.ts` and `.tsx`.
JS aliases	same, non-relative specifiers	Match tsconfig `compilerOptions.paths` patterns (`*` wildcard) first, then `baseUrl`; resolve as above

Source	Collected from	Resolution
Python absolute	`import_statement`, `import_from_statement`	`a.b.c` → `a/b/c.py` or `a/b/c/__init__.py` under each of `python_source_roots` (default: scan root and `src/`). For `from a.b import c`, also try `a/b/c.py`, since `c` may be a submodule.
Python relative	`from . import x`, `from ..pkg import y`	Walk up one folder per leading dot from the importer's folder, then resolve as above

A path is counted once however many times it's imported. The file never counts itself. Package imports that resolve to nothing in the scan set are ignored. The resolved target list is kept for §10.2 but isn't serialized.

9.4 Effective lines (A4)

Every rule that used lines now uses effective lines, so packing code into long lines no longer shrinks a file.

- A line is effective if it has at least one character outside whitespace and comments. Comment nodes are kind `comment`. Python docstrings (a string as the first statement of a module, class or function) count as comments.
- Each effective line counts `ceil(chars / effective_line_width)`, where `chars` is its length with surrounding whitespace removed and the width defaults to 100. A 450-character line counts as 5.
- `effective_lines_altered` applies the same rule to only the altered lines (§11).
- Per function: the effective lines inside its range, excluding named functions nested in it.
- `line_count` stays physical (newlines, plus 1 for a final line without one) and is for display only.

10 · Flag rules, ratings & the purity score (`engine/config.py`, `engine/run.py`)

The three master rules stay: concentration (§4.1), semantic path (§4.2) and fragmentation (§4.3), combined into one 0–100 score (§4.4) and a success/failure conclusion (§4.5). The review changes what they measure, not what they are. Every rule now uses effective lines, the score is weighted by altered lines, and both fail-closed paths count as flagged.

10.1 Config, skips and fail-closed files (A3, C4)

```
# argon/engine/config.py
from pydantic import BaseModel

class EngineConfig(BaseModel):
    # master rules
    line_threshold: int = 800                    # §4.1, effective lines
    async_boundary_threshold: int = 3            # §4.1
    semantic_path_flag_threshold: float = 40     # §4.2, PLACEHOLDER until
calibration
    nesting_flag_threshold: float = 40           # §10.4, PLACEHOLDER until
calibration
    rating_mode: str = "master"                  # "master" | "nesting"
    fragmentation_avg_import_threshold: float = 4
    fragmentation_min_files: int = 3
    fragmentation_flat_penalty: int = 25
    purity_score_fail_threshold: int = 40        # §4.5
    # review fixes
    effective_line_width: int = 100              # A4
    max_error_coverage: float = 0.05             # C2
    max_file_bytes: int = 1_000_000              # C4
    score_cap: int = 1_000_000                   # B3
    count_optional_calls: bool = False           # A2
    vendored_path_globs: tuple[str, ...] = ("**/node_modules/**",
"**/dist/**", "**/build/**",
        "**/.next/**", "**/coverage/**", "**/vendor/**")
    generated_path_globs: tuple[str, ...] = ("**/*.d.ts",
"**/__generated__/**",
        "**/*_pb2.py", "**/*_pb2_grpc.py", "**/*.generated.*")
    generated_markers: tuple[str, ...] = ("@generated", "DO NOT EDIT", "auto-
generated")
    test_path_globs: tuple[str, ...] = ()        # §6.1 defaults
    python_source_roots: tuple[str, ...] = (".", "src")
```

```
    ts_path_aliases: dict[str, list[str]] = {}   # filled by the CLI from
tsconfig
    ts_base_url: str | None = None
    # async boundary lists: §9.2 defaults

DEFAULT_CONFIG = EngineConfig()
```

`should_skip_file(filename, cfg) -> SkippedFile | None` skips a file only by **path**:

- Matches `vendored_path_globs` → skipped, reason `vendored_path`.
- Matches `generated_path_globs` → skipped, reason `generated_path`.
- An in-file marker (`@generated` and the rest, in the first 5 lines) **never** causes a skip on its own (A3). The file is scored, and its name is added to `ignored_generated_markers` so a person can decide whether to add its path to `generated_path_globs`. In Phase 1, these globs must come from the base branch's config, never the PR's own, so a PR can't exempt itself.

Files larger than `max_file_bytes` aren't parsed (C4). Unless a skip glob matches them, they are flagged `oversize`. Files above `max_error_coverage` are flagged `unparseable` (§5.2). Both count as flagged, take 30 overshoot points, and use non-blank lines (with the width rule) as their weight, since there is no syntax tree to find comments.

10.2 Fragmentation (A8, A9)

Built on the internal import graph (§9.3), not a PR-wide average, so isolated padding files can't dilute it.

1. Nodes are the analyzed files; each resolved internal import is a directed edge.
2. Find the largest group of files connected by imports, ignoring direction.
3. Triggered when that group has at least `fragmentation_min_files` files and its edges ÷ files is greater than `fragmentation_avg_import_threshold`.
4. Applied once as the flat 25-point penalty. It never sets any file's `flagged` (master §4.4). The group's files go in `fragmentation_cluster`.

10.3 Purity score (A5, B2, B6)

Each file *f* has weight *w* = `effective_lines_altered`. If every weight is 0, all weights are 1. *T* is the active threshold.

```
\text{flag\_ratio} = \frac{\sum_f w_f \cdot [\text{flagged}_f]}{\sum_f w_f}
```

```
\text{pts}_f = \begin{cases} 0 & \text{rating}_f \le T \\ \min\!\left(30,\ 10 \log_{10}(\text{rating}_f / T)\right) & \text{otherwise} \end{cases}
```

```
\text{score} = 100 - 70 \cdot \text{flag\_ratio} - \frac{\sum_f w_f \cdot \text{pts}_f}{\sum_f w_f} - 25 \cdot [\text{fragmentation}]
```

The score is clamped to 0–100 and rounded with `floor(x + 0.5)`. No files analyzed gives 100. A rating 10× over the threshold costs 10 points, 100× costs 20, and 1,000× or more costs the full 30. Trivial padding files carry almost no weight, so they no longer dilute the ratio.

10.4 Ratings (A11, B1)

Ratings are computed **per function**, using each function's own counts. Module-level code is rated as one pseudo-function, `<module>`. A file's `logic_complexity_rating` is the highest rating among its functions, for the active `rating_mode`.

```
\text{rating\_master}_f = \frac{\text{branches}_f^{\,2} \times \text{effective\_lines\_altered}_f}{\text{outputs\_effective}_f}
```

```
\text{rating\_nesting}_f = \frac{\text{score}_f \times \text{effective\_lines\_altered}_f}{\text{outputs\_effective}_f}
```

`rating_master` is master §4.2's formula, applied to one function instead of a whole file. That stops long files of simple functions from being rated as one giant nest (B1). `rating_nesting` replaces the squared branch count with the nesting score. That score already grows quadratically for a deep nest ($1 + 2 + \ldots + n$) but only linearly for a flat switch, which is what the squaring was meant to do. Both are computed and reported for every function. The gate uses `rating_mode`, `"master"` by default, until calibration (§11) shows which separates bloat from normal code better.

10.5 Rules and conclusion

Function	Rule
`check_concentration_rule(f, cfg)`	`f.effective_lines > line_threshold and f.async_boundary_count > async_boundary_threshold`

Function	Rule
`check_semantic_path_rule(f, cfg)`	`f.logic_complexity_rating > active threshold`
`check_fragmentation_rule(findings, edges, cfg)`	§10.2
`compute_purity_score(findings, fragmentation, cfg)`	§10.3
`determine_check_conclusion(score, cfg)`	`"failure"` if `score < purity_score_fail_threshold`, else `"success"`
`build_summary_markdown(result)`	master §6.1 template, plus skipped files, ignored markers and the fragmentation group
`run_local(files, altered=None, cfg)`	skip → parse → `analyze_tree` → per-file rules → fragmentation → score → conclusion → summary. Pure; no I/O.

`run_local` takes `files` as a map from POSIX path to bytes, and `altered` as a map from path to the set of altered line numbers. With no `altered`, every line counts as altered. In Phase 1, master's `async def run(job)` fetches the PR's files and diff, then calls `run_local`.

11 · CLI (`cli.py`)

One command scans files or folders and prints the file table, the function ranking and the PR-comment preview. It exits non-zero on failure, so it can run as a plain CI step before the GitHub App exists. All disk and `git` access lives here.

```
python -m argon.cli <path> [<path> ...]
    [--top N]               # function ranking rows (default 20, 0 = none)
    [--base REF]            # score only files and lines changed since REF
    [--config FILE.toml]    # override any EngineConfig field
    [--json]                # print EngineResult as JSON, nothing else
    [--calibrate]           # add the calibration report
    [--calibrate-out FILE.csv]  # one row per function, for analysis
```

Flag	Behavior
paths	Files or folders, walked recursively for supported extensions. Paths are POSIX, relative to the first path if it's a folder, else the current folder. Symlinks are not followed, and files whose real path is outside the scan root are dropped (C5).
`--base REF`	Runs `git diff -w --unified=0 REF -- <paths>` once. Only files in the diff are analyzed, and altered lines are the `+` line numbers of each hunk. `-w` ignores whitespace-only edits (A5). New untracked files count as fully altered.
no `--base`	Every file is analyzed and every line counts as altered.
`--config`	TOML whose top-level keys are `EngineConfig` fields. Unknown keys are an error.
config discovery	If `tsconfig.json` or `jsconfig.json` exists at the scan root, its `compilerOptions.paths` and `baseUrl` fill `ts_path_aliases` and `ts_base_url` (A8). Comments in those files are allowed (JSONC).
`--top N`	Prints the N highest-scoring functions across all files, with the §8.6 columns plus both ratings.
`--json`	`EngineResult.model_dump_json(indent=2)`.

Exit codes: `0` success, `1` failure, `2` usage error or nothing to analyze.

Default output, in order:

1. Warnings: parse errors under the limit, ignored generated markers, saturated scores.
2. The file table, sorted by rating: file, effective lines, altered, branches, nesting score, depth, outputs, async, internal imports, rating, flag reason.
3. The function ranking (`--top N`).
4. Skipped files with their reasons, and the fragmentation group if it fired.
5. `summary_markdown`, exactly as the PR comment would read:

```
### Argon Structural Review -- Purity Score: 78/100

| File | Issue | Rule |
|---|---|---|
| src/orders/handler.ts | 940 effective lines, 4 async boundaries |
Concentration Rule |
| src/utils/validate.ts | Logic complexity 52 (threshold 40) in `validate()`
| Semantic Path Logic |

2 of 14 changed files flagged.
```

Calibration report (`--calibrate`). This is how the placeholder thresholds get replaced with real ones.

- p50, p75, p90, p95, p99 and max for function-level `rating_master`, `rating_nesting`, `score`, `branches`, `max_depth` and `effective_lines`.
- For each threshold in 40, 100, 300, 1,000, 3,000 and 10,000: the share of functions and of files each rating mode would flag.
- The purity score the scan would get under each rating mode.
- With `--calibrate-out`, one CSV row per function with every metric, so two repos (one hand-written, one AI-generated) can be compared side by side.

12 · Tests, verification & definition of done

The MVP is done when every test below passes, the parity checks hold, and the calibration report has been run on at least three real repos. Expected scores come from the worked examples in *How Argon scores code* or follow by hand from §6–§8. Purity scores were computed with code.

12.1 Scoring fixtures

Fixture	Code (abridged)	Nesting score	Flat	Other
`nested_ifs.py`	`if a:` → `if b and c:` → `if d:`	7	4	reported depth 3, `nested`
`isolation_1.py`	§8 first example	outer 3, inner 1	2	

Fixture	Code (abridged)	Nesting score	Flat	Other
`isolation_2.py`	§8 second example	outer 12, inner 3	4	
`comprehension.py`	`[f(x) for a in b if c for x in a]`	6	3	equals the unrolled loop
`loop_header.py`	`for r in (p for p in ps if p): pass`	1 + 1 + 2 = 4	3	the comprehension sits at the loop's depth, not inside it; its if-clause is one deeper
`elif_chain.js`	`if (a) {} else if (b) {} else if (c) {}`	3	3	all at depth 0
`ternary_chain.js`	`a ? b : c ? d : e` / `a ? (b ? x : y) : z`	2 / 3	2 / 2	
`loop_else.py`	`for …: pass` + `else: pass`	2	2	
`match.py`	`case 1 if g:`, `case 2:`, `case _:`	2	2	guard and catch-all not counted (B10)
`except_nest.py`	two `except` at depth 0, an `if` inside one	1 + 1 + 2 = 4	3	`try` adds no depth
`control_flow.js`	`ok && (b && (c && run()))`	6	3	A7
`guard_chain.js`	`a && b && c && run()`	3	3	A7: only the outer operator is control flow
`closure_nest.js`	`if (a) { xs.forEach(x => { if (b) { if (c) {} } }) }`	1 + 3 × 2 = 7	3	reported depth 3, `nested` (A6)
`test_asserts.py`	10 `assert x and y` in `tests/`	0	0	B9

Fixture	Code (abridged)	Nesting score	Flat	Other
`deep_callbacks.js`	generated: 40 callbacks, each inside an `if`	capped at 1,000,000	40	`saturated` (B3)
chaos fixture, JS twin	not yet defined (open question 4)	3		required by the reference

12.2 Loophole regression tests

Each compares a gamed file with its honest twin, so the tests hold whatever the thresholds become.

Test	Gamed version	Must hold
A1 padding	the same function plus 20 unreachable `return "pN"` lines	identical `outputs_effective` and ratings
A1 cap	a 1-decision function with 6 distinct returns placed after fail() (a call that always throws, which the dead-code rule can't see)	`outputs_effective` = 2
A3 marker	`// @generated` on line 1, path not in `generated_path_globs`	file scored, listed in `ignored_generated_markers`
A4 packing	the same code joined onto as few lines as possible	effective lines within 30% of the formatted version
A5 dilution	one 200-line file rated 4,000 (flagged) plus 19 one-line files rated 5	score 18; unweighted it would be 96
A8 Python imports	`pkg/a.py` with `from pkg.b import x` and `import pkg.c`	`relative_import_count` = 2
A8 TS alias	tsconfig `"@/*": ["src/*"]`, `import x from "@/b"`	resolves to `src/b.ts`
A9 dilution	6 files that all import each other, plus 20 isolated files	fragmentation still fires; the group is the 6

Test	Gamed version	Must hold
C1 missing	unclosed `{` (TS), unclosed `(` (JS), `def f(:` (Python)	`has_error` true for all three
C2 fail closed	a file with error coverage above 5%	flagged `unparseable`, counted as flagged
C3 encoding	bytes `\xff\xfe` in a `.py` file	no exception
C4 oversize	a 2 MB `.js` outside skip globs	flagged `oversize`, not parsed

Score math (no parsing). Build `FileFinding`s directly with *T* = 40:

- Equal weights of 10, ratings 75 (flagged), 10 and 20 → 75.757 → **76**, `success`. With fragmentation → **51**.
- One file rated 40,000,000 → **0** (overshoot capped at 30, flag ratio 1).
- No findings → **100**. A raw score of 46.5 rounds to **47** (B6).
- Concentration at 801 effective lines and 4 boundaries → flagged; 800 or 3 → not.
- Fragmentation never changes any file's `flagged` or `flag_reason`.

12.3 Parity and independent checks

Carried over from the reference as required checks:

- **Flat counts** match radon 6.0.1 on CPython's standard library plus installed packages (the reference used 14,052 files), apart from the deliberate differences in §6.4. They match ESLint's `complexity` rule on JS/TS (2,505 files), excluding default parameters and `?.`.
- **Nesting scores** match an independent reference on the same Python files. That reference takes radon's per-construct increments and applies §7–§8 over Python's own `ast` module, separately from tree-sitter.
- The review fixes change both checks: A7 and the test-assert rule (B9) alter some scores. Parity runs with A7 off and outside test paths, and the fixes are covered by §12.2.

12.4 Done when

- [ ] All tests in §12.1–12.3 pass
- [ ] A test proves `engine/` imports no I/O modules

- [ ] CLI exits 1 on a failing fixture folder and 0 on a passing one; `--json` output validates back into `EngineResult`
- [ ] 1,000 files scan in under 30 seconds on a laptop
- [ ] `--calibrate-out` run on three repos (one hand-written, two AI-generated); thresholds and `rating_mode` chosen and written into `config.py` with the data saved in `calibration/`

13 · Open questions

1. **Thresholds and rating mode.** Both rating thresholds are placeholders at 40, and the gate uses `rating_mode="master"` until calibration. The decision: after the §11 calibration runs, which mode separates AI-generated bloat from normal code better, and at what threshold?
2. **Fragmentation threshold.** Imports are counted once per target file, so an average above 4 needs at least 6 files that nearly all import each other. Master's PR-wide average had the same limit. Is that strict enough, or should the threshold drop to 2–3 after calibration?
3. **Async dedupe.** This spec counts one boundary per receiver (§9.2), not per call-site as master's text says. Confirm.
4. **Chaos fixture.** The reference requires `tests/fixtures/chaos.py` and its JS twin to score 3, but neither source says what they contain. Supply them or define them.
5. **What counts as altered.** Added lines only, so deleting code is never penalized. The alternative is added plus deleted.
6. **Crashed workers in Phase 1 (D3).** This spec completes the Check Run as `failure`, which blocks a required check until someone re-runs it. `neutral` would let the merge through instead. Which do clients expect?